

02. LSTM & GRU

Overview

LSTM and GRU cells add gates that regulate what enters, stays in, and leaves memory. We derive the full update equations, compare parameter budgets, and cover bidirectional and stacked variants used in modern encoders. The focus is on why a separate cell state fixes gradient paths that plain tanh RNNs cannot maintain.

Lab: Lab 02.

Prior modules: Module 01.

Contents

1	Limits of Vanilla RNNs	2
2	LSTM Cell Equations	2
2.1	Gradient Highway	2
3	GRU: A Lighter Alternative	3
4	Bidirectional and Stacked Variants	3
5	Implementation Notes	3
6	Lab and Forward Links	3
7	Peephole and Layer Normalization	4
7.1	Gradient Norm Monitoring	4
8	Extended Intuition	4
8.1	Gradient Paths Through the Cell	4
9	Numerical Walkthrough	4
10	PyTorch Implementation Notes	5
11	Local GPU Constraints	5
12	Debugging Checklist	5
13	Practice Problems	6

Module track

This module builds on **Module 01**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

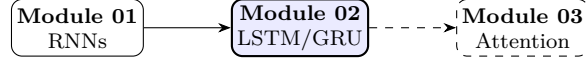


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

Recap from Module 01

Vanilla RNNs backpropagate through a product of Jacobians:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{i=t-k+1}^t \mathbf{W}_{hh}^\top \text{diag}(\phi'(\mathbf{z}_i)). \quad (1)$$

When $\|\mathbf{W}_{hh}\| < 1$ and $\phi = \tanh$, the norm shrinks exponentially with lag k . The LSTM cell (Section 2) adds a separate memory path controlled by gates.

1 Limits of Vanilla RNNs

Equation (1) restates the BPTT product from **Module 01**: gradients through k shared steps shrink when spectral norms stay below 1. Vanilla RNNs therefore struggle with long-range dependencies. **LSTM** [3] and **GRU** [1] introduce gating to control information flow.

Note. Gates are sigmoid activations in $(0,1)$ that implement differentiable write/read/forget operations on memory.

2 LSTM Cell Equations

At time t , given input $\mathbf{x}_t$ and previous hidden/cell $(\mathbf{h}_{t-1}, \mathbf{c}_{t-1})$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate}) \quad (2)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate}) \quad (3)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate}) \quad (4)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{candidate}) \quad (5)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (6)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (7)$$

The cell state $\mathbf{c}_t$ acts as a conveyor belt; $\mathbf{f}_t \approx 1$ preserves information across many steps.

2.1 Gradient Highway

Because $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathbf{f}_t$ (element-wise), gradients can flow with minimal attenuation when forget gates are open.

3 GRU: A Lighter Alternative

GRU merges cell and hidden state, using reset gate $\mathbf{r}_t$ and update gate $\mathbf{z}_t$:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t]), \quad (8)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t]), \quad (9)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t]), \quad (10)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (11)$$

GRU has $\approx 75\%$ of LSTM parameters for the same hidden size.

Table 1: LSTM vs. GRU trade-offs.

Criterion	LSTM	GRU
Separate cell state	Yes	No
Parameter count	Higher	Lower
Long sequences	Slightly better	Competitive
Training speed	Slower	Faster

4 Bidirectional and Stacked Variants

A bidirectional LSTM runs forward and backward encoders:

$$\vec{\mathbf{h}}_t = \text{LSTM}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}), \quad (12)$$

then concatenates $[\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ for rich context, used heavily in **Module 09** (BERT architecture).

Stacking L layers feeds layer ℓ 's $\mathbf{h}_t^{(\ell)}$ as input to layer $\ell+1$.

5 Implementation Notes

- Initialize forget bias to 1 to encourage memory retention early in training.
- Use dropout on non-recurrent connections (between layers), not on recurrent links within a step.
- ‘batch_first=True’ layout: (B, T, d) simplifies DataLoader integration.

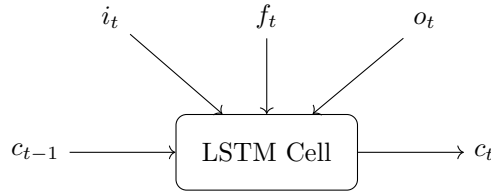


Figure 2: LSTM gate inputs to cell update (6).

6 Lab and Forward Links

Lab 02 trains LSTM/GRU classifiers on sequential data; compare validation accuracy and training time using table 1. Attention mechanisms in **Module 03** (attention definition) reduce reliance on fixed-size $\mathbf{h}_T$ bottlenecks from **Module 01** (sequence output layouts).

7 Peephole and Layer Normalization

Peephole LSTM adds cell-state connections to gates: $f_t = \sigma(W_f[h_{t-1}; x_t] + W_{fc}c_{t-1})$. LayerNorm on recurrent outputs stabilizes training:

$$\text{LN}(\mathbf{h}) = \gamma \odot \frac{\mathbf{h} - \mu}{\sigma} + \beta. \quad (13)$$

7.1 Gradient Norm Monitoring

Track $\|\nabla_{\theta}\mathcal{L}\|_2$ each epoch; spikes often precede NaN losses in long sequences.

8 Extended Intuition

LSTM and GRU cells exist because vanilla RNNs multiply gradients through $\mathbf{W}_{hh}$ at every step. A separate cell state $\mathbf{c}_t$ (LSTM) or a blended hidden update (GRU) creates paths where information can flow with near-constant magnitude. Gates are not magic; they are learned switches implemented with sigmoids that output values in $(0, 1)$.

The forget gate decides how much of $\mathbf{c}_{t-1}$ survives. The input gate decides how much new candidate content enters the cell. The output gate decides what portion of the cell state becomes the visible hidden state $\mathbf{h}_t$. GRU merges forget and input into an update gate and uses a reset gate to control how much past context influences the candidate. Fewer parameters than LSTM, often similar accuracy on NLP benchmarks under 512 hidden units.

Stacking layers ($L = 2$ or 3) lets higher layers capture phrase-level patterns while lower layers track local syntax. Bidirectional encoders concatenate forward and backward final states; use them for classification, not autoregressive generation. For generation you need causal (left-to-right) recurrence only, as in **Module 10**.

8.1 Gradient Paths Through the Cell

Backprop through an LSTM step adds gradients to $\mathbf{c}_{t-1}$ through the forget gate without passing through tanh on the full cell. That additive path is why $\mathbf{f}_t \approx 1$ lets gradients travel hundreds of steps while vanilla RNNs decay exponentially. The tradeoff: if the model learns $\mathbf{f}_t \rightarrow 0$ everywhere, the cell resets every step and you are back to a vanilla RNN with extra parameters. Watch gate saturation in logs: mean $\mathbf{f}_t$ stuck near 0 or 1 across batches often precedes plateauing validation loss. LayerNorm inside the cell (as in some speech models) stabilizes gate pre-activations when input embeddings are poorly scaled.

9 Numerical Walkthrough

Single-layer LSTM: $d_x = 300$ (GloVe), $d_h = 256$, $T = 50$, $B = 16$, 2-class sentiment.

Gate pre-activations at step t use one fused linear map:

$$\begin{bmatrix} \mathbf{i}_t \\ \mathbf{f}_t \\ \mathbf{g}_t \\ \mathbf{o}_t \end{bmatrix} = \mathbf{W} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{h}_{t-1} \end{bmatrix} + \mathbf{b}, \quad (14)$$

where $\mathbf{W} \in \mathbb{R}^{4d_h \times (d_x + d_h)}$. With $d_x = 300$, $d_h = 256$: fused weight matrix has shape $(1024, 556)$, about 569k parameters per layer.

Cell update (with $\mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t$ after sigmoid, $\mathbf{g}_t = \tanh(\cdot)$):

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (15)$$

Suppose $\mathbf{f}_t \approx 0.9$, $\mathbf{i}_t \approx 0.1$ early in training: the cell mostly preserves prior content with small injections. After 10 epochs on 25k reviews, validation accuracy might move from 78% (bag-of-words baseline) to 86% with dropout 0.3 on the final $\mathbf{h}_T$.

GRU with same dims has $\mathbf{W} \in \mathbb{R}^{3d_h \times (d_x + d_h)}$, roughly 25% fewer params than LSTM. Expect similar step time on GPU because both are memory-bandwidth bound at small batch sizes.

Gradient check at step $t = 25$ with $\mathbf{f}_t = 0.95$, $\mathbf{i}_t = 0.05$: $\partial \mathcal{L} / \partial \mathbf{c}_{t-1}$ retains roughly 0.95 of the upstream signal before the input gate branch. Compare to vanilla RNN where $\|\partial \mathbf{h}_t / \partial \mathbf{h}_{t-k}\| \propto \|\mathbf{W}_{hh}\|^k$; with $\|\mathbf{W}_{hh}\| = 0.9$ and $k = 50$, that factor is $0.9^{50} \approx 0.005$. Peephole connections (optional, rarely used in modern stacks) let gates read $\mathbf{c}_{t-1}$ directly; PyTorch `nn.LSTM` does not expose them, but the idea clarifies why the cell state is the memory bus.

10 PyTorch Implementation Notes

- `nn.LSTM(input_size, hidden_size, num_layers=2, dropout=0.3, batch_first=True)`

Applies inter-layer dropout on outputs of layers $0..L - 2$, not on the last layer output.

- Returns `(output, (h_n, c_n))`; `h_n` and `c_n` have shape `(num_layers, B, d_h)`.
- `nn.GRU` drops the cell state; API otherwise mirrors LSTM.
- `bidirectional=True` doubles d_h in `output` (concat forward/backward); adjust classifier input dim to $2d_h$.
- CuDNN requires contiguous tensors and fixed-length batches unless packed; set `enforce_sorted=False` when lengths are unsorted.

```
lstm = nn.LSTM(300, 256, num_layers=2, dropout=0.3, batch_first=True)
packed = pack_padded_sequence(x, lengths, batch_first=True, enforce_sorted=False)
out_packed, (h_n, c_n) = lstm(packed)
out, _ = pad_packed_sequence(out_packed, batch_first=True)
# many-to-one: h_n[-1] has shape (B, 256)
```

11 Local GPU Constraints

Sentiment LSTM with $d_h = 256$, $T = 128$, $B = 32$, 2 layers fits in under 2 GB with Adam. Push to $d_h = 512$ or $T = 256$ and memory climbs quickly because LSTM stores four gate tensors per step. On 4 GB, use $B = 16$, accumulate gradients over 2 to 4 steps for effective batch 32 to 64.

Mixed precision helps multi-layer stacks; wrap the forward pass in `autocast` and use `GradScaler`. If training on CPU for debugging, set `num_layers=1` and $T = 32$ to keep iteration time under a few seconds per batch.

CuDNN LSTM fuses gate computations when `torch.backends.cudnn.enabled=True`; disable only when debugging numeric mismatches against a manual cell implementation. Weight decay on biases is usually omitted; AdamW with `weight_decay=0.01` on LSTM weights only is a common pattern in classification fine-tunes. Layer-wise lr decay (lower layers $0.95 \times$ lr per level) helps when fine-tuning pretrained encoders; less critical on random-init LSTM labs.

12 Debugging Checklist

- Identical train/val loss but chance-level accuracy: forgot to use `h_n[-1]` from the last layer, or took mean pooling without a mask.
- Bidirectional LSTM on generation task: impossible autoregressive setup; switch to unidirectional for decoding.

- Initial loss $\approx \log(2)$ on binary task but never improves: labels swapped or `CrossEntropyLoss` given one-hot instead of class indices.
- Slower after enabling `bidirectional=True`: expected; two passes over the sequence per layer.
- Hidden state explosion: reduce learning rate or add gradient clipping; check for unnormalized word embeddings.
- `c_n` not carried across batches: each new batch should reset `(h_0, c_0)` unless you intentionally preserve state for streaming inference.
- Dropout on final `h_T` only: applying dropout inside recurrence without `LockedDropout` duplicates noise patterns incorrectly across time.

13 Practice Problems

Problem 1. Count LSTM parameters for one layer with $d_x = 100$, $d_h = 128$. Include biases.

Hint: Four gates, each with weights for $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$.

Problem 2. Why does setting $\mathbf{f}_t \rightarrow 1$ and $\mathbf{i}_t \rightarrow 0$ approximate an identity carry for $\mathbf{c}_t$?

Problem 3. Train identical sentiment models with LSTM and GRU; report val accuracy and wall-clock time per epoch.

Problem 4. Implement bidirectional encoding and concatenate $\vec{\mathbf{h}}_T$ and $\overleftarrow{\mathbf{h}}_1$ for classification.

Problem 5. Log mean forget-gate activation per epoch during sentiment training. Correlate drops in $\mathbb{E}[\mathbf{f}_t]$ with validation accuracy stalls.

Problem 6. Stack depth $L = 3$ with dropout 0.3: compare trainable params and peak VRAM to $L = 1$ at the same $d_h = 256$.

Compare results in **Lab 02**; attention in **Module 03** removes the need to squeeze all context into one $\mathbf{h}_T$.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 02.

Topic	Primary source	Where in this PDF
LSTM gate equations	[3]	Section 2
GRU update equations	[1]	Section 3
LSTM variant survey	[2]	Section 7
Gradient stability context	[6]	Eq. 1

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

- [1] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*, 2014.
- [2] Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, et al. LSTM: A search space odyssey. In *IEEE Transactions on Neural Networks and Learning Systems*, 2017.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *ICML*, 2015.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [6] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013.